

RTOS 기본 개념

신인철

Contents

RTOS 기본 개념

Real-Time OS 기초

Real-Time OS 기본 개념

Real Time OS 기초

Real Time OS 기초

□ 실시간 운영체제란?

○ 운영체제가 없던 시절?

- 계산기 프로그램을 수행시켜 계산을 하려한다고 하자. 계산은 CPU 에서 이루어지지만 이에 앞서 CPU 가 처리할 명령들이 메모리에 적재되어 있어야 한다. 또한 결과를 얻어서 출력을 하기 위해서는 프린터로 결과를 보내야 한다. 즉, 여러 가지 자원이 있지만 이를 쓰기 위해서 무언가 이를 관리해주는 프로그램이 필요하다. 이런 자원을 이용하는 여러 개의 프로그램을 작성할 때 각각의 프로그램마다 별도로 자원을 관리하는 기능을 넣는다면 이는 매우 비효율적인 일이 될 것이다.
- 예를 들어 계산기 프로그램 내부에 load 하는 기능까지 넣을 수도 있겠지만 load 하는 프로그램은 계산기뿐 아니라 다른 모든 프로그램에서 공통이 되기 때문에 따로 loader 라는 프로그램을 둔다. 이처럼 운영체제는 여러 프로그램에서 공통적인 부분을 묶어서 시스템 프로그램을 작성하면서부터 시작됐다.
- 예전에 컴퓨터라는 것은 매우 값비싼 장비였다. 따라서 컴퓨터를 효율적으로 쓰는 것이 매우 중요했는데, 가령 어떤 프로그램이 CPU 와 관련된 작업을 끝내고 I/O(예를 들어 프린터)와 관련된 작업을 시작했을 때 CPU 는 다른 일을 하지 않고 대기 상태에 있었다. 이를 해결하기 위해서 multiprogramming 이란 개념이 도입되었다.
- 그러나 여러 사람이 컴퓨터를 쓰는 경우에 다른 사람은 쓸 수 없었기 때문에 한번 사용하기 위해 많은 시간을 기다려야 했다. 이를 해결하는 방법은 시간을 나누어서 여러 사람들 모두를 만족시키는 것이었다. 즉, 여러 사람에게 조금씩 시간을 할당해 주었고 한 사람 한 사람은 마치 자신이 혼자 컴퓨터를 쓰는 것처럼 느끼게 했다. 이를 time-sharing 이라고 한다.
- 이처럼 여러 사람이 동시에(논리적으로) 서로 제 각각 프로그램들을 수행하게 되었고 서로 충돌이 나지 않도록 해야 하는 기능이 필요하게 되었다. 그래서 task 들이 충돌이 나지 않도록 하는 monitor program 이 생겨났고 이것이 점점 발전되어 운영체제의 일부가 되었다. 이와 같은 과정을 거치면서 multiprocessing(multitasking), task scheduling 등의 개념이 OS 에서는 일반화가 되었다.

Real Time OS 기초

□ 실시간 운영체제란? (계속)

○ 운영체제의 역할

- 요즘에 일반적인 OS 가 하는 일을 살펴보면 CPU 에서 수행되는 프로그램들을 관리하기 위한 task management(task scheduling, communication, synchronization 등을 포함) memory 를 관리하기 위한 memory management, I/O 를 관리하기 I/O management, file system management 등이 있다. 이런 작업들은 앞으로 계속 다룰 real-time OS 에서도 공통적으로 하는 일이다.

Real Time OS 기초

□ 실시간 운영체제가 등장한 이유?

○ Multiprocessor 시스템 제어를 위해

- Multiprocessor system은 여러 개(multi)의 processor 들이 한 system 에 들어가서 동시에 작업을 수행하는 것이다.
- (multiprocessing 과는 다른 개념) 이때 여러 개의 processor 들을 관리하기 위해서 기존의 OS 가 아닌 multiprocessor 용 OS 가 필요하다. 최근 들어 그러한 용도로 real-time OS 가 쓰이고 있으나 이 과정에서는 multiprocessor 에 대한 내용은 다루지 않는다.

Real Time OS 기초

□ 실시간 운영체제가 등장한 이유?(계속)

○ Embedded 시스템 제어를 위해

- Embedded system 이라는 것은 일반적인 computer system 과는 달리 특정한 작업만을 하도록 설계되며 초기의 embedded system 은 비교적 단순해서 OS 가 필요 없이 인간이 순차적인 프로그램을 작성해서 수행하도록 했고 중간에 interrupt 가 발생하는 경우에만 그 순차적인 프로그램에서 잠시 벗어나는 정도였다. 그러나 최근 들어 multimedia 정보를 처리해야 하는 embedded system 이 늘어나면서 그 system 이 해야 할 일들도 많아지고 복잡해 졌기 때문에 순차적인 프로그램 작성이 매우 어렵게 되었다.
- 따라서 embedded system 에서 OS 의 개념이 필요하게 되었으며 embedded system 의 특성상 real-time 이라는 요소를 만족해야 했다. 따라서 real-time OS 가 embedded system에 도입된 것이다.
- 참고적으로 real-time 시스템에 대해 설명하자면 정해진 시간 제약을 해결할 수 있는 시스템을 말한다. 이 말은 주어진 시간 내에 어떠한 일을 반드시 처리해야 한다는 말이다. 즉 단지 빨리 처리해야 하는 것이 아니고 정해진 시간을 넘겨서는 안된다는 것이다. Real-time 에서 가장 대표적인 분야로는 군사용을 들 수 있다. 예를 들어 미사일이 어떤 목표를 추적함에 있어서 어떤 계산이 주어진 시간 내에 이루어지지 않는다면 그 미사일은 그 목표를 격추시킬 수 없는 것이다. 우리가 쓰는 컴퓨터에서는 editor 가 어느때는 좀 빨리 어느때는 좀 늦게 실행된다고 해서 크게 문제가 될 건 없다. 그러나 embedded system 에서 주어진 입력중에는 시간내에 처리하지 못하면 문제가 될 수 있기 때문에 real-time 요소를(보통 soft real-time)만족시켜야 한다.

Real Time OS 기초

□ 실시간 운영체제와 기존의 운영체제와의 차이점 (2가지)

○ 실시간성 충족

- 여러 가지 방안이 있겠으나 대체로 효율성, 속도(좀더 엄밀히 말하면 시간 제약), 공정성 (fairness)등을 들 수 있겠다. 일반적인 OS에서는 자원(예를 들어, 메모리, 하드 디스크 등등)을 얼마나 효율적으로 낭비 없이 쓸 것인가에 초점이 맞춰져 있다. 그러나 real-time OS에서는 효율적인 것도 중요하지만 그 속도에 좀 더 신경을 쓴다.
- 보통 C 프로그램을 작성할 때에 메모리를 할당하기 위해서는 malloc 이라는 함수를 쓰게 된다. 이 함수는 malloc 을 할 수 있는 영역에서 메모리를 찾게 된다. 만약 메모리가 당장 쓸 수 없는 경우에는 기다리는 일도 발생된다. 그러나 real-time OS 에서 메모리를 할당하는 경우에는 malloc 을 쓰지 않고 real-time OS 에서 제공하는 메모리 할당 함수를 쓴다. 이 함수가 malloc 과 차이점이 있다면 정해진 구역(이미 처음에 메모리를 할당하기 위한 메모리 풀을 선언하게 된다)에서 메모리를 할당하려고 시도하고 만약에 당장 할당할 메모리가 없다면 무작정 기다리는 것이 아니라 현재 할당할 수 없음을 알리게 된다. 이는 무작정 메모리 할당을 기다리다가 정해진 시간 제약을 만족하지 못해서 real-time 으로 실행하 지 못하게 됨을 막기 위한 것이다.

○ 우선순위 고려 사항

- UNIX 와 같이 여러명의 사용자가 쓰는 경우에는 각 사용자들이 실행하는 프로그램이 task 로서 수행이 되고 대개의 경우에는 우선순위에 많은 차이를 두지 않는다. 이것은 그 task 들 사이에 공정성을 유지하려 하기 때문이다. 그러나 real-time OS 에서 task 는 대개 우선순위가 차이가 있도록 하며 이때 task 사이의 공정성은 고려하지 않는 것이다.
- 이상에서 본 것처럼 메모리를 할당하거나 task 를 scheduling 하는 작업은 두가지 OS 에서 모두 필요한 작업이므로 작업 자체는 공통적인 것으로 볼 수 있다.

Real Time OS 기본 개념

Real Time OS 기본 개념

□ Task와 multitasking

○ Multitasking 이란?

- UNIX를 써본 적이 있는 사람은 여러 사람이 한꺼번에 login 을 해서 동시에 자신의 프로그램들을 수행할 수 있음을 알 것이다.
- 굳이 UNIX 를 예로 들지 않더라도 Windows 시스템인 경우만 해도 Sound card를 통해서 음악을 들으며 C compiler 는 compile 을 수행하면서 printer 로 문서를 출력할 수 있음을 알고 있다. 이때 음악 연주 프로그램, compiler program, printer spooler를 각각의 task라고 한다.
- 그리고 이처럼 여러 개의 task 를 동시에(여기에서 말하는 의미는 어떤 기간을 말하는 것으로 어느 한 시점으로 보면 아님, multiprocessor system 과 비교!) 실행할 수 있는 것을 multitasking 이라 한다.

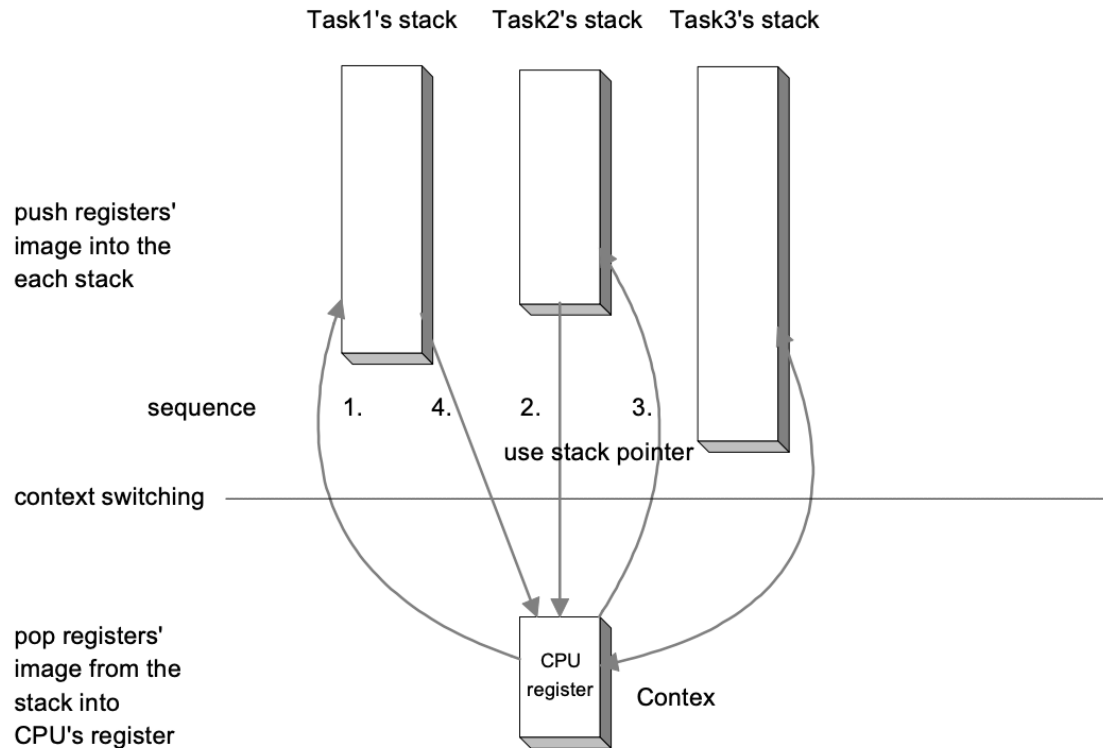
○ Embedded 시스템에서 multitasking

- Embedded system 에서도 처리할 작업이 많아지면서 multitasking 을 적용하게 되었으나 앞에서 말한 것과는 차이가 있다.
- 일반적인 컴퓨터에서는 각 task 들이 대부분 무관한 프로그램들이지만 embedded system 에서의 task 들은 하나의 큰 응용 프로그램을 논리적으로 나눈것이다.
- 즉, 어떤 응용 프로그램이 수행되기 위해서는 여러 기능들이 동시에(물리적으로는 아님) 수행되어야 할 필요가 있고 이를 순차적으로 프로그램을 하기 어렵기 때문에 사용자가 프로그램을 쉽게 하기 위해서 도입한 개념이 바로 multitasking 이다. 하나의 응용 프로그램을 논리적으로 나누었기 때문에 기능상 매우 밀접한 관계가 있기 때문에 task 사이에 이루어지는 작업들이 매우 많다.

Real Time OS 기본 개념

□ Context switching (문맥교환)

○ Multitasking과 문맥교환



Real Time OS 기본 개념

□ Context switching (문맥교환) (계속)

○ Multitasking과 문맥교환 (계속)

- 우선, task1 에서 하던 작업이 중단되고 task2 가 수행되려 한다고 가정하자.(이렇게 되는 경우는 매우 많.) task1 이 완전히 종료가 된 상태라면 별 문제 없겠지만 중단된 것이기에 task2 가 수행을 마치거나 중단되는 경우에 다시 task1 을 수행하게 된다면 이전에 수행되던 상태를 알아야 할 것이다.(이는 함수에서 함수를 다시 호출하는 경우와 비슷하다.)
- 따라서 task2 가 수행되기 이전에 task1 의 상태를 저장해 둘 필요가 있다.
- Computer system 에서 상태라는 것은 Register, 메모리, 하드 디스크 등에 저장된다. 이때 메모리 같은 자원은 task 마다 나누어 쓰면 별 문제가 없을 수 있지만 CPU 의 register 는 공유하는 자원이므로 구역을 나누어 쓸 수 없다. 따라서 상태를 저장해야 하는 가장 중요한 자원은 register 이다. 그래서 task1 이 중단되기 전에 register 값들을 task1 의 stack 에 보관을 하고 task2 가 수행되고 다시 task1 이 수행될 시점이 오면 stack 에 저장되었던 값들을 register 로 옮겨와서 task1을 다시 수행하게 된다.

Real Time OS 기본 개념

□ Kernel

○ 기본 역할

- OS 란 앞에서도 언급했지만 컴퓨터 자원을 효율적이고 쉽게 쓰도록 하기 위한 소프트웨어이다. OS 의 기능 중에서도 핵심이 되는 부분을 kernel 이라고 하고 여기에서는 앞에서 설명한 context switching, task scheduling, memory management 등의 작업을 한다.

○ Scheduler

- multitasking 환경에서 다음에 어떤 task 가 수행될 것인지 결정하는 책임을 맡는다. kernel 이라고 부르는 부분에서도 핵심적인 부분이라고 할 수 있는 것이 바로 scheduler로서 dispatcher 라고도 한다.
- 일반 multitasking OS 에는 여러 가지 scheduling 기법에 있으며 real-time OS 에도 이에 맞는 여러 가지 scheduling 기법들이 제안되었다. 그러나 구현상의 문제 등으로 실제로 대부분의 real-time OS 에서 쓰는 scheduling 기법은 priority-based scheduling 이다. 또한 같은 priority에 대해서는 FIFO 나 round-robin 방법이 적용된다.

Real Time OS 기본 개념

□ Kernel (계속)

○ Non-Preemptive Kernel

- 우리말로 비-선점형 kernel 이라고 한다. 쉽게 말해서 kernel 의 능력이 크지 못하다고 할 수 있다. 즉, 어떤 task 가 수행되고 있을 때 kernel 이 중간에서 그 task 의 수행을 중지시키고 다른 task 를 수행시킬 수 있는 능력이 없다. 다른 말로 cooperative multitasking 이라고도 한다. 그래서 preemptive kernel 보다 design 하기 쉽다.
- Preemptive 되는 경우는 오직 Interrupt Service Routine(ISR)에 의해서이다. 이때도 역시 ISR에서 interrupted task 로 제어권을 수동으로 넘겨 줘야 한다.
- 장점으로 interrupt latency 가 비교적 짧다는 것이고 또 하나 re-entrant function 을 쓸 수 있다는 것이다. 그러나 단점은 task 가 제어권을 넘기지 않으면 kernel 이 아무 지시도 내릴 수 없기 때문에 task 들이 매우 잘 계획이 되어야 한다는 것이다. 따라서 priority 가 높은 task 도 무한정 기다릴 수 있기 때문에 real-time system 에서는 일반적으로 잘 쓰이지 않는다.

Real Time OS 기본 개념

□ Kernel (계속)

○ Preemptive Kernel

- 우리말로 선점형 kernel 이라고 한다. 이 kernel 은 어떤 task 를 중단시키고 다른 task 를 수행시킬 능력이 있는 것이다. 따라서 앞의 non-preemptive kernel 보다는 구현하기 복잡하다.
- 또한 interrupt latency 가 조금 길어진다는 단점이 있다.
- 그러나 이런 단점에도 불구하고 대부분의 real-time OS 에서 채택하는 이유는 priority 가 높은 task 가 먼저 수행될 수 있다는 점 때문이다. 즉, 가장 priority 가 높은 task 는 가장 먼저 수행될 수 있다는 deterministic 한 성격을 지닌다.
- 앞의 non-preemptive kernel 에서는 ISR 에서 interrupted task 로 제어권을 넘겨줘야 했지만 이 경우에 ISR 이 interrupted task 보다 높은 priority task 를 호출한다면 ISR 이 끝나고 interrupted task 로 진행되는 것이 아니고 priority task 로 진행이 되고 interrupted task 는 suspend 상태가 되는 것이다. 앞으로 설명할 OS 는 이 preemptive kernel 이다.

Real Time OS 기본 개념

□ Kernel (계속)

○ Critical Section

- 다른 task 에 의해서 중단되어서는 안되는 일련의 명령 혹은 코드의 블록을 말한다. 예를 들어 shared memory 를 접근하는 부분에서는 중단이 되는 경우에는 그 결과를 보장할 수 없다.
- 이를 해결하기 위해서는 아래에서 설명하는 mutual exclusion 이 보장되어야 한다.

○ Mutual Exclusion

- 예를 들어서 두개의 task 가 동시에 printer 를 쓰려고 한다고 가정하자. 만약 task1 이 프린트 하는 도중에 task2 가 실행되고 task2 가 실행되는 도중에 다시 task1 이 실행된다면 나온 결과는 task1 이나 task2 모두 원하지 않는 결과일 것이다. 이때 mutual exclusion 이 되어야 할 필요가 있다. 또한 위에서 언급한 shared memory 도 마찬가지이다. 즉 shared resource 를 쓰는 경우에는 mutual exclusion 이 보장되어야 하는 것이다.
- 이를 해결하기 위해서는 몇가지 방법이 있는데 첫째 가장 쉬운 방법은 interrupt 가 발생하는 것을 막는 것이다. 어떤 task 에서 다른 task 로 제어권이 넘어가는 경우의 한가지로 자신이 message 를 기다리는 상태로 가는 상황을 생각해 볼 수 있는데 이런 것을 shared resource를 쓰는 부분에서는 제외시킬 수 있다. 이런 상황을 막는다고 하면 남는 것은 round-robin 에 의해서 다른 task 로 제어권이 넘어가는 경우나 interrupt 가 발생하는 경우다. Round-robin 에서 다른 task 로 제어권을 넘기는 것 역시 timer(clock) interrupt 가 발생한 경우이므로 한 마디로 interrupt 를 막으면 된다. 즉, critical section 으로 들어가기 전에 interrupt 를 disable 시키고 빠져 나오면서 interrupt 를 enable 시킨다.
- 이외에 semaphore 라는 mechanism 이 이용된다.(semaphore 에 대해서는 아래에서 설명) semaphore 를 얻지 못하면 shared resource 를 얻을 수 없으며 일단 semaphore를 얻으면 shared resource 를 마음 놓고 쓸 수 있다. 다른 task 를 위해서 shared resource를 다 쓰면 semaphore 를 풀어야 한다.

Real Time OS 기본 개념

□ Kernel (계속)

○ Semaphore

- Semaphore 는 Edsger Dijkstra 가 60 년대 중반에 고안한 mechanism 으로서 대부분의 multitasking kernel 에서 쓰이고 있다. Semaphore 는 공유 변수로서 앞에서 설명한 mutual exclusion 을 만들어 공유 resource 를 제어할 수도 있고 task 사이에 synchronize를 하기 위해서도 이용된다.
- Shared resource 를 쓰는 경우를 예로 들어보면 우선 semaphore 를 만들고 shared resource 를 쓰기 직전에 semaphore 를 얻는다. 얻는다는 의미는 변수값이 0 이 아닌 양수인 경우 변수값을 1 을 감소시키고 계속 진행하는 것이며 만일 0 이라면 누군가 다른 task 에 의해 값이 양수로 변하기를 기다리고(누군가 shared resource 를 다 쓰면 그 값을 1 증가 시킬 것이다) 양수가 되면 그때 역시 1 을 감소시키면서 계속 진행한다.
- Semaphore 가 0 과 1 만이 될 수 있는 것을 binary semaphore 라고 하고 그 이상의 값을 갖는 것을 counting semaphore 라고 한다.

Real Time OS 기본 개념

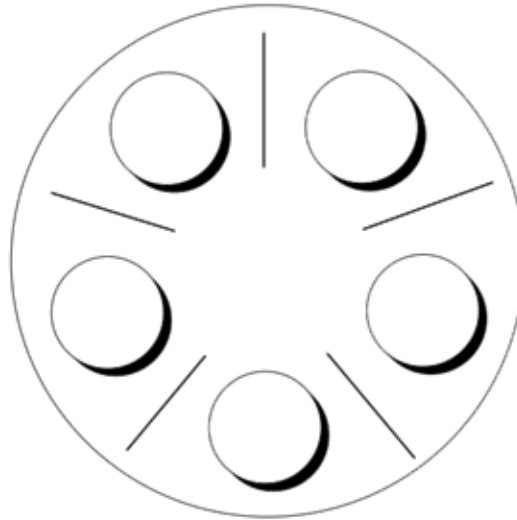
□ Kernel (계속)

○ Deadlock

- deadlock 은 시스템이 더 이상 진행될 수 없는 상태에 도달하는 것을 말한다. Dijkstra 가 예로 든 ‘저녁식사하는 5 명의 철학자’이야기가 있다. 5 명의 철학자가 식탁에 앉아서 식사를 한다. 포크는 모두 5 개가 있다. 음식은 스파게티인데 매우 미끄러워 2 개가 필요하다. 철학자들은 각각 생각을 하다가 먹고 다시 또 생각한다. 이를 프로그램으로 시뮬레이션할 때 계속해서 진행되도록 할 수 있는가?
- 이때 자신의 왼쪽 포크를 잡으면 오른쪽 포크를 찾아서 식사를 하게 한다. 만약 모두 왼쪽 포크를 들고 오른쪽 포크를 찾는다면? (프로그램으로 생각해본다면 철학자 한사람이 왼쪽을 들고 오른쪽을 들기전에 context switch 가 일어나서 다른 사람이 왼쪽을 들고, 다시 context switch 가 일어나고..) 이 사람들은 모두 자신의 오른쪽에 있는 포크가 내려질때까지 기다릴 것이고 그런 상황은 언제까지나 오지 않을 것이다. 이는 서로 남이 소유한 자원을 얻으려고 하기 때문에 일어난다. 이제 이를 조금 바꿔서 자신의 오른쪽 포크가 없다면 왼쪽 포크를 내려놓고 얼마간 기다린 후 다시 왼쪽 포크를 집도록 바꿔 보자. 만약 왼쪽 포크를 동시에 들었다면, 동시에 내려 놓을 것이고 얼마간 지나서 다시 동시에 다시 왼쪽 포크를 들 것이다.

Real Time OS 기본 개념

- Kernel (계속)
- Deadlock (계속)



```
#define N      5
philosopher(int i)
{
    while (TRUE)
    {
        think();
        take_fork(i);
        task_fork((i+1) % N);
        eat();
        put_fork(i);
        put_fork((i+1) % N);
    }
}
```

Real Time OS 기본 개념

□ Kernel (계속)

○ Priority Inversion

- 3 개의 서로 다른 priority 를 갖는 task 를 가정해보자. 이때 $\text{priority}(\text{task1}) > \text{priority}(\text{task2}) > \text{priority}(\text{task3})$ 라고 가정하자. 현재 task1 과 task3가 ready 상태라고 가정하자. Task1 이 가장 priority 가 높기 때문에 먼저 수행될 것이다.
- task1 과 task3 가 하나의 semaphore 를 쓴다고 가정하자. 만약 task3 가 semaphore 를 갖고 있으면 priority 가 높은데도 불구하고 task3 가 수행이 될 것이다. 만약 task3 가 곧 semaphore 를 release 해서 task1 이 수행된다면 이 상황은 큰 문제가 안될수도 있겠지만 만약 task3 가 수행도중에 task2 에 의해 제어권을 뺏기는 경우 task2 가 task1 보다 priority 가 낮은데도 불구하고 먼저 수행된다.

Real Time OS 기본 개념

□ Kernel (계속)

○ Task Communication

- task 간에 communication 하는 방법으로는 global data 를 쓰는 것과 message passing 의 2 가지 방법이 있다.
- Global variable 을 쓰는 경우에 exclusive access 를 해야하는데 ISR 이 포함된 interrupt 를 disable 해야하며 task 사이에서는 interrupt 를 disable 하는 방법 외에 semaphore를 쓸 수도 있다.
- Message passing 의 방법으로는 mailbox, queue, pipe 등이 있다. 이 세가지는 message 의 size에 따라 나눌 수 있다.

	Size	number
mailbox	4 X 32 bit	1
queue	m X 32bit	n
pipe	m X 8bit	n

Real Time OS 기본 개념

□ Kernel (계속)

○ Task Synchronization

- task 사이에서 순서를 정할 때 쓰인다. 보통 semaphore 를 쓸 수도 있고 event flag 나 signal 을 쓸 수도 있다. 예를 들어 어떤 task 가 I/O 를 초기화 한후 semaphore 를 기다린다.
- ISR 에서는 I/O operation 을 끝내고 semaphore 를 통해 알려주므로써 그 task 가 다시 resume 될수 있도록 하는 역할을 한다.
- Event flag 는 Event 가 여러개 발생했을 때의 synchronize 를 위해서 필요하다. Nucleus 에서는 Event group 이라는 용어로 쓰인다. 이 group 에서는 한 bit 마다 특정 event 에 대응해서 어떤 event 가 발생되면 그 bit(event flag)가 set 되며 이때 그 event 를 기다리는 task 가 ready 상태가 된다.
- signal 이란 event flag 와 비슷하지만 차이가 있다. signal 이 발생되면 그 task 는 interrupt 되고 특정 signal handling routine 이 호출된다. 이는 마치 interrupt service routine 의 경우와 비슷하다.

Real Time OS 기본 개념

□ Kernel (계속)

○ Interrupt Service

- interrupt 는 asynchronous event 를 CPU 에 알리는 방법이다. Interrupt 는 외부에서 들어오는 중요한 신호로서 시간에 민감한 경우가 있기 때문에 interrupt latency 가 짧은 것이 좋다.
- 또한 ISR 자체도 짧은 것이 좋은데 그 이유는 ISR 자체가 길어지면 interrupt nesting 이 되기 쉽기 때문이다. 따라서 ISR 에서는 보통 그에 상응하는 task 수준의 service routine 을 부르고 (HISR) 끝나도록 구성을 한다. HISR 에서는 마치 task 처럼 존재해서 수행이 된다.

○ soft real-time vs. hard real-time

- real-time system 중에는 어떤 시간안에 반드시 처리해야 하며 그 시간이 지난 후의 결과 값은 정확해도 소용이 없는 경우가 있다. 이런 시스템을 hard real-time system 이라고 한다. 이에 반해서 어떤 시간안에 처리하면 좋지만 그렇지 못한 경우에 그 시간에서 약간 경과한 후의 값도 인정하는 경우를 soft real-time system 이라고 한다. 여기에서 설명되는 것은 soft real-time system 이다.

Real Time OS 기본 개념

□ Kernel (계속)

○ reentrancy

- reentrant code란 말 그대로 코드가 다시 진입할 수 있음을 나타낸다. 어떤 함수가 reentrant라는 것은 task1에서 function1 함수를 수행하다가 도중에 task2로 제어권이 넘어가고 이 함수를 다시 호출해도 function1이 제대로 동작한다는 것이다.
- 어떤 code를 reentrant하도록 하는 방법은 전역변수를 쓰지 않는 것이다. 전역변수라는 것은 shared resource로 볼 수 있으므로 mutual exclusive하도록 만들어 주지 못하면 문제가 생기는 것이다.
- 그렇지 않은 code(function)는 공유하지 않거나 꼭 공유해야 하는 경우에는 semaphore를 쓴다거나 동일 priority를 갖는 task 사이에서 round-robin을 하지 않아야 한다.